ENGINEERING WORKBENCH

Common technical assignment | R&D; Internship 2026-2027

Candidate ID: **RA2311003020323**
Report type: prototype progress and technical design summary
Candidate repository: github.com/gittarunsg/Pass-Science-Works---Assessment

Scope note: this report describes the C++ source prototype and analytical reference data. Compiler-based build and test results are not included.

Executive summary

This package contains a C++17 command-line prototype for one-dimensional steady thermal FEA, input validation, a deterministic parallel batch path, a single-sample neural-network gradient, and simulated embedded-device adapters. It also contains synthetic sample data and analytical reference outputs. The source and report were prepared with substantial AI assistance; see the AI-use disclosure in the project archive.

Assessment area	Current evidence
Thermal solver	C++ source and analytical convergence reference; executable verification not run.
CAD / nodal data	ASCII STL facet counter and synthetic CSV parser/sample; no 3D visualization UI.
Parallel / embedded / NN	Source paths and mock interface design; no measured benchmark or physical-device evidence.
Delivery	CMake project and editor configuration; no compiled Launch.exe or one-click package.

Verification scope

CMake and a C++ compiler were not available in the authoring environment. Therefore the C++ project and authored test suite have not been built or run. No standalone launch, passing test results, or candidate-machine measurements are reported.

1. Thermal finite-element formulation

The rod uses two-node linear elements with constant properties and insulated lateral surfaces. For an element of length h, the stiffness contribution is (kA/h) [[1,-1],[-1,1]], and the consistent source vector is (qAh/2)[1,1]. Prescribed endpoint temperatures are imposed by eliminating their contributions from the interior equations.

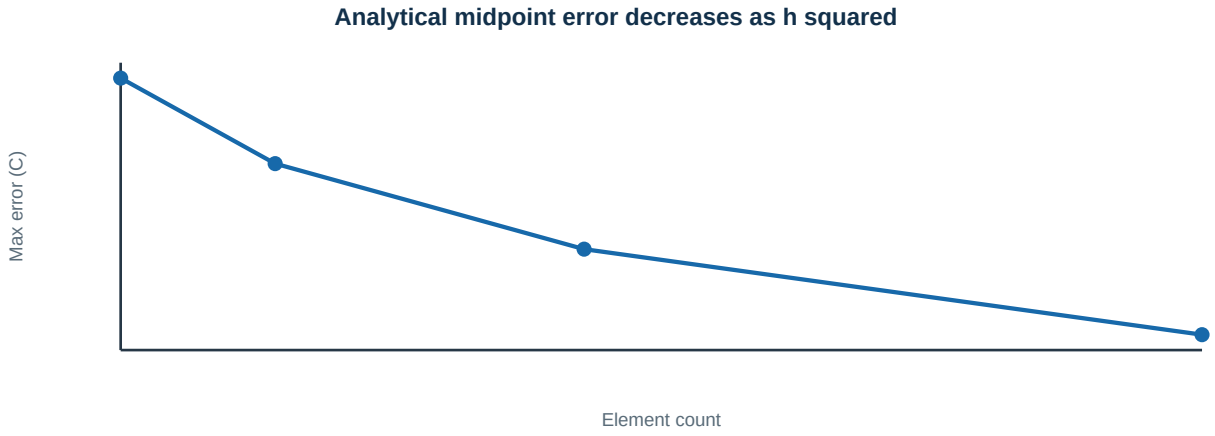
Problem values and exact solution

L=1 m; A=1e-4 m^2; k=10 W/(m K); q=10,000 W/m^3; T(0)=100 C; T(L)=20 C. The exact field is T(x)=100-80x+500x(1-x) C, with x in metres.

Midpoint interpolation error

For this quadratic exact field, the piecewise-linear interpolation error at an element midpoint is q h^2/(8k). These are analytical references from the equation, not values read from a compiled run.

Elements	Max midpoint abs. error (C)
10	1.25000000
20	0.31250000
40	0.07812500
80	0.01953125



Verification note: test source checks endpoint values, convergence values, finite-difference gradients, parallel agreement and mock-device inventory. Those checks remain unexecuted until a C++17 compiler and CMake are available.

2. Software structure and sample data

The project separates numerical and parsing functions in src/engine.cpp from the command-line adapter in src/main.cpp. CMakeLists.txt declares C++17 and standard-library threads, with no third-party runtime dependency. The required interactive multi-view application is not present; the current interface is command-line only.

Supported prototype inputs

Input	Included	Limits
ASCII STL	Self-created tetrahedron sample; source counts ASCII facets.	No triangle rendering, camera controls, binary STL, scale metadata, or file-size study.
Temperature CSV	Synthetic point sample; parser checks exact header, row width, parseability and finite values.	No colored 3D point cloud, legend, node picking, ANSYS export claim, or file-size study.

Data provenance

The STL is self-created with arbitrary units because STL carries no scale metadata. The CSV is generated from the analytical rod curve and is explicitly synthetic, not an ANSYS export. Provenance notes and a deterministic reference-data script are included in the archive.

CLI source inventory

Commands are defined for FEA CSV export, STL/CSV inspection, deterministic batch solving, NN gradient/loss export, and mock-device reporting. These are source-level descriptions until compiled and run.

Candidate-specific work remaining

Implement the four-view UI, validation, progress and safe cancellation; add 3D interactions and result export; verify malformed-file cases and practical file limits; capture actual application screenshots and logs.

3. Parallel and embedded integration

Parallel workload design

A fixed seed generates independent cases with positive conductivity and generation. A bounded worker pool claims case indices under a mutex and writes to stable result slots, preserving deterministic order. The CLI compares parallel values with a serial batch. It currently reports one timed run per worker setting; it does not yet provide the prescribed five repetitions, median, machine profile, efficiency, warm-up protocol, or separate launch-to-ready time.

Benchmark status

Measurement	Status
CPU / cores / RAM	Not recorded; candidate-machine measurement required.
Compiler / flags / workload	Not recorded; no compiler available here.
Five repetitions / median / speedup	Not run; benchmark template is blank in the archive.
Serial-parallel max delta	Test threshold is 1e-8 C; result not observed.

Embedded integration design

Mocks represent an MCU board, FPGA board and JTAG debug adapter. A 3.3 V MCU I/O domain and 1.8 V FPGA bank are illustrative assumptions only. Grounding, power, connectors and signal levels must be checked for exact hardware. A JTAG adapter owns debug signals such as TCK/TMS/TDI/TDO as applicable; it is a debug interface, not a compute board. Mock telemetry and a simulated timeout do not demonstrate physical connectivity.

Before physical connection

Identify exact board models and revisions, verify vendor pinout and voltage-tolerance documents, check power direction, and determine level translation and ground requirements. This package does not recommend wiring based only on its illustrative assumptions.

4. Backpropagation and verification notes

Single-sample derivation

For x=1 and y=2: h=tanh(wx+b), y_hat=vh+c, e=y_hat-y, and J=0.5e^2. Initial values: w=0.5, b=0, v=1, c=0, learning rate=0.1.

Forward pass: h=tanh(0.5)=0.462117; y_hat=0.462117; e=-1.537883; J=1.182542.

Chain rule: dJ/dv=e*h=-0.7108; dJ/dc=e=-1.5379; dJ/dw=e*v*(1-h^2)*x=-1.2095; dJ/db=e*v*(1-h^2)=-1.2095.

Simultaneous update using original gradients: w=0.62095, b=0.12095, v=1.07108, c=0.15379. Negative initial gradients make the parameters increase after subtracting learning-rate times gradient.

The archive contains C++ gradient logic and finite-difference test source. The reference loss curve is mathematically generated; it is not executable evidence. The required legible handwritten derivation must be produced by the candidate; this report does not imitate handwriting or claim the appendix exists.

Analytical reference: loss during 100 gradient steps

Evidence included and remaining

Included: C++ source, CMake project, authored test source, synthetic sample inputs, analytical reference plots, and embedded-interface assumptions. Remaining evidence: compiler build and CTest output, an interactive UI and packaged Windows launcher, five-repetition machine benchmark, candidate-created handwritten pages, and demonstration screenshots or video. The benchmark and analytical references are labeled according to how they were produced.